FD Rates Tracker

Complete Code Explanation

A guide to every Python module, database table, template, scraper, and API route in this project.

Stack: FastAPI + Jinja2 + MySQL (Hostinger) + live bank scrapers

Banks: Yes Bank, HDFC, CUB, Axis, IDFC, ICICI, SBI, IndusInd

1. What this project does

This app scrapes Fixed Deposit (FD) interest rates from official Indian bank websites, stores them in MySQL, archives each scrape as JSON files, and shows them on a CUB-style dashboard. You can also compare best rates across banks and export CSV/Excel.

Accuracy rule

Only rates published by the bank are shown. If a bank does not publish a particular tenure/slab combination, the cell shows a dash (-). The app never guesses or copies rates from another slab.

Standard table format (CUB style)

- FD types: Callable and Non-Callable
- Deposit slabs: <3 crore and 3 Cr < 5 Cr
- Tenures: 365-443 days, 444 days, 445-554 days, 555 days, 556 days to 3 Years

2. Architecture (how requests flow)

When you open the dashboard or click Refresh Rates, this is the flow:

```
Browser
 -> main.py (FastAPI routes)
  -> services.py (business logic / table building)
   -> scrapers.py + rate_scrapers.py (fetch from bank sites)
   -> db_storage.py (save/read MySQL)
   -> scrape_archive.py (save JSON under data/scrapes/)
 <- templates/*.html (HTML response)
```

File map

- main.py - HTTP routes and app startup/shutdown
- config.py - settings loaded from .env
- bank_data.py - bank names, source URLs, tenure rows
- schemas.py - Pydantic models for API responses
- services.py - scrape orchestration + dashboard compare logic
- scrapers.py - converts raw scraper dicts into RatePoint objects
- rate_scrapers.py - one live scraper per bank
- db_storage.py - MySQL connection and queries
- scrape_archive.py - timestamped JSON archives
- schema.sql - MySQL table definitions
- templates/ - Jinja2 HTML pages

3. config.py - Settings

Uses pydantic-settings. Values come from .env (MYSQL_HOST, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DATABASE, etc.). Important flags:

- INIT_DB_ON_STARTUP=false - do not run schema every start (Hostinger limits)
- SCRAPE_ON_STARTUP=false - scrape only when you click Refresh

- SCRAPES_DIR - folder for archived scrape sessions
- EXCEL_EXPORT_FILE - where Excel export is written

4. bank_data.py - Master bank metadata

Defines TENURE_ROWS (the 5 standard tenure bands), DEPOSIT_SLABS, and BANKS. Each bank has a slug (e.g. hdfc), display name, and source_urls for callable and non_callable pages/PDFs.

Helpers: get_source_url(slug, fd_type) returns the official URL shown under each table; all_bank_slugs() returns the bank order for the UI.

5. schemas.py - Data shapes

- BankOut - slug + name for APIs
- RatePoint - one scraped cell (fd_type, tenure, slab, rate, source_url). Used when writing into MySQL.
- RateOut / HistoryOut / BestRateOut - JSON response models

6. schema.sql - Database tables

banks

Primary key: slug. Stores bank display name.

fd_rates (current rates)

One row per bank + fd_type + tenure_label + deposit_slab (unique). Holds the latest rate_percent, tenure day range, sort_order, source_url, and updated_at.

rate_history

Append-only log of rate changes. When a scrape writes a new value that differs from the previous one, a history row is inserted.

7. db_storage.py - MySQL layer

All database access goes through this file. Hostinger shared MySQL has a strict connection-per-hour limit, so the code keeps ONE reused connection (_get_connection) protected by a threading lock.

Key functions

- init_db() - runs schema.sql
- ensure_banks_exist() - inserts any missing bank rows
- get_rates() / get_dashboard_data() - read current rates
- clear_bank_rates(slug) - delete that bank's current rates before refresh
- upsert_rates_and_record_history(slug, points) - save new rates and write history when values change
- export_to_excel() - builds Excel with openpyxl
- close_db() - called on app shutdown

8. rate_scrapers.py - Live bank scrapers

This is the largest and most important file. Each bank has a scrape_*() function that downloads HTML/PDF/JSON from the official source and maps rates onto the CUB tenure rows.

Shared helpers

- _fetch(url) - HTTP GET with browser-like headers
- _fetch_rendered(url) - Playwright for JS-heavy pages (if needed)
- parse_tenor_range(text) - convert '444 days' / '1 year to 3 years' into day min/max
- TenorBand - holds one bank tenor band and rates for slabs
- pick_rate(bands, day, slab) - choose the narrowest tenor band that covers a representative day (e.g. day 444 for the '444 days' row)
- to_cub_format / to_cub_from_slab_tables - map bank bands into the standard dict shape: {callable: {tenure: {slab: rate}}, non_callable: ...}

Per-bank methods

- HDFC - HTML interest-rates tables (<3 Cr and >=3 Cr, non-withdrawable)
- CUB - HTML deposit-interest-rate page (callable + non-callable, both slabs)
- Axis - PDF via pdfplumber (separate callable / FD Plus PDFs)
- IDFC - retail + bulk PDFs discovered from interest-rate page, HTML fallback
- IndusInd - rates.html tables, classified by premature-withdrawal headers
- ICICI - official JSON feed (fd-interest-rate.json)
- SBI - retail HTML tables (callable table 0, non-callable table 1)
- Yes Bank - CMS content API for callable <3 Cr; bulk PDF for 3-5 Cr callable and non-callable

SCRAPERS dict maps slug -> function. scrape_bank(slug) runs the matching scraper.

9. scrapers.py - RatePoint adapter

LiveBankScraper.fetch_rates() calls rate_scrapers.scrape_bank(), then walks TENURE_ROWS and converts each published slab into a RatePoint. This is the object format db_storage expects.

10. scrape_archive.py - Local JSON history

Every scrape creates data/scrapes/<timestamp>/. Each bank gets rates.json. summary.json lists success/fail and points_seen. Useful for debugging without querying MySQL.

11. services.py - Business logic

scrape_banks()

1) Ensure banks exist in MySQL. 2) Create archive session. 3) For each bank: clear old rates, fetch live rates, upsert + history, save JSON. 4) On failure, clear that bank's rates and store the error. 5) Write summary.json.

build_bank_tables()

Groups DB rates into CUB-style HTML tables. Missing slabs become '-'. Rows with both slabs empty are skipped. Empty whole tables are skipped.

build_best_rate_comparison() / build_ranked_rates()

Used by /compare. For each tenure, finds the highest rate per slab across banks, then builds full rankings.

12. main.py - FastAPI routes

Lifecycle

- startup - optional init_db / scrape_banks (both off by default)
- shutdown - close MySQL connection

Pages

- GET / - dashboard (optional ?bank_slug=hdfc)
- GET /compare - best-rate comparison (?fd_type=callable|non_callable)
- POST /scrape/run-page - Refresh Rates button, then redirect to /

APIs / exports

- GET /banks, GET /rates, GET /rates/compare/best
- POST /scrape/run - JSON scrape for selected banks
- GET /export/csv, GET /export/excel

13. templates/ - UI

- base.html - layout, CSS, nav (Home, Best Rates, Refresh, Export)
- index.html - bank chips + CUB rate tables + source links
- compare.html - best rate table + full ranking per tenure/slab

14. End-to-end: clicking Refresh Rates

Step by step:

- Browser POSTs /scrape/run-page
- services.scrape_banks() loops all scrapers
- Each scraper downloads official page/PDF/JSON and maps to CUB tenures
- scrapers.py turns mapped dict into list[RatePoint]
- db_storage clears old rows for that bank, inserts new rates, logs changes
- JSON archived under data/scrapes/<timestamp>/
- Redirect to /?scraped=1
- dashboard reads MySQL via get_dashboard_data() and renders tables

15. How to run

```
cd C:\Users\pratik.bhatkulkar\Desktop\FD
python -m uvicorn main:app --host 127.0.0.1 --port 8000

Open: http://127.0.0.1:8000
```

Ensure .env has MYSQL_* credentials. Keep INIT_DB_ON_STARTUP=false and SCRAPE_ON_STARTUP=false on Hostinger to avoid connection-limit errors. Starlette must stay compatible with FastAPI 0.115.5 (starlette>=0.40,<0.42).

16. Why some cells show '-'

This is expected accuracy behavior, not a broken URL. Examples:

- Yes Bank non-callable: bulk PDF has 3-5 Cr only (no <3 Cr)

- HDFC/Axis/IDFC non-callable <3 Cr at long tenors often not published
- SBI retail page has limited non-callable tenors and no 3-5 Cr slab

If a bank publishes a rate you can see on their site but the app shows '-', that specific scraper/mapping needs a fix for that bank+tenure+slab.